

Script Criação:

```
-- =====
-- PRE-REQUISITO: Populando campos faltantes da tabela PRODUTO
-- O INSERT original nao preencheu valor_unitario,
-- quantidade_atual e quantidade_minima.
-- Este UPDATE e necessario para a Situacao 6 funcionar.
-- =====

UPDATE produto SET
    valor_unitario = CASE id_produto
        WHEN 1 THEN 7999.99 WHEN 2 THEN 4999.99 WHEN 3 THEN 4599.00
        WHEN 4 THEN 8999.00 WHEN 5 THEN 9999.00 WHEN 6 THEN
11999.00
        WHEN 7 THEN 3999.00 WHEN 8 THEN 899.00 WHEN 9 THEN 5999.00
        WHEN 10 THEN 1299.00 WHEN 11 THEN 2499.00 WHEN 12 THEN 1999.00
        WHEN 13 THEN 4299.00 WHEN 14 THEN 3499.00 WHEN 15 THEN 2999.00
        WHEN 16 THEN 2799.00 WHEN 17 THEN 799.00 WHEN 18 THEN 1099.00
        WHEN 19 THEN 999.00 WHEN 20 THEN 2199.00
    END,
    quantidade_atual = CASE id_produto
        WHEN 1 THEN 15 WHEN 2 THEN 3 WHEN 3 THEN 0 WHEN 4 THEN
8
        WHEN 5 THEN 5 WHEN 6 THEN 20 WHEN 7 THEN 1 WHEN 8 THEN
0
        WHEN 9 THEN 12 WHEN 10 THEN 7 WHEN 11 THEN 25 WHEN 12 THEN
4
        WHEN 13 THEN 6 WHEN 14 THEN 10 WHEN 15 THEN 3 WHEN 16 THEN
9
        WHEN 17 THEN 30 WHEN 18 THEN 2 WHEN 19 THEN 50 WHEN 20 THEN
5
    END,
    quantidade_minima = CASE id_produto
        WHEN 1 THEN 5 WHEN 2 THEN 5 WHEN 3 THEN 2 WHEN 4 THEN
3
        WHEN 5 THEN 5 WHEN 6 THEN 4 WHEN 7 THEN 3 WHEN 8 THEN
1
        WHEN 9 THEN 5 WHEN 10 THEN 3 WHEN 11 THEN 10 WHEN 12 THEN
3
        WHEN 13 THEN 4 WHEN 14 THEN 5 WHEN 15 THEN 5 WHEN 16 THEN
5
```

```

        WHEN 17 THEN 10    WHEN 18 THEN 5    WHEN 19 THEN 20    WHEN 20 THEN
3
    END;

-- =====
-- SITUACAO 1
-- FUNCAO: fn_total_venda(p_id_venda INT)
-- Retorna o valor total de uma venda somando todos os
-- itens do carrinho (quantidade * valor_unitario).
-- Usa COALESCE para retornar 0.00 em venda sem itens.
-- =====

CREATE OR REPLACE FUNCTION fn_total_venda(p_id_venda INT)
RETURNS DECIMAL(15,2)
LANGUAGE plpgsql
AS $$
DECLARE
    v_total DECIMAL(15,2);
BEGIN
    SELECT COALESCE(SUM(quantidade * valor_unitario), 0.00)
    INTO    v_total
    FROM    carrinho_venda
    WHERE   fk_id_venda = p_id_venda;

    RETURN v_total;
END;
$$;

-- TESTES - Situacao 1
SELECT fn_total_venda(1) AS "Total Venda 1   (Carlos Silva)";
SELECT fn_total_venda(4) AS "Total Venda 4   (Julia Ferreira)";
SELECT fn_total_venda(99) AS "Total Venda 99 (Inexistente)";

-- Relatorio: todas as vendas com totais calculados
SELECT
    v.id_venda,
    c.nome           AS cliente,
    v.data_venda,
    fn_total_venda(v.id_venda) AS total_calculado
FROM venda v
JOIN cliente c ON c.id_cliente = v.fk_id_cliente
ORDER BY total_calculado DESC;

```

```

-- =====
-- SITUACAO 2
-- FUNCAO: fn_endereco_completo(p_id_cliente INT)
-- Retorna o endereco formatado de um cliente como texto unico,
-- concatenando logradouro, numero, complemento (opcional),
-- bairro, cidade e CEP. Usa JOIN com cidades.
-- =====

CREATE OR REPLACE FUNCTION fn_endereco_completo(p_id_cliente INT)
RETURNS TEXT
LANGUAGE plpgsql
AS $$
DECLARE
    v_endereco TEXT;
BEGIN
    SELECT
        e.tipo_logradouro || ' ' || e.nome_logradouro || ', ' ||
e.numero
        || CASE WHEN e.complemento IS NOT NULL
            THEN ' - ' || e.complemento
            ELSE ''
        END
        || ' - ' || e.bairro
        || ' - ' || ci.nome
        || ' - CEP: ' || e.cep
    INTO v_endereco
    FROM enderecos e
    JOIN cidades ci ON ci.id_cidade = e.fk_id_cidade
    WHERE e.fk_id_cliente = p_id_cliente
    LIMIT 1;

    RETURN COALESCE(v_endereco, 'Endereco nao cadastrado');
END;
$$;

-- TESTES - Situacao 2
SELECT fn_endereco_completo(1) AS "Endereco Cliente 1 (Carlos
Silva)";
SELECT fn_endereco_completo(2) AS "Endereco Cliente 2 (Ana Souza)";
SELECT fn_endereco_completo(10) AS "Endereco Cliente 10 (Diego
Rodrigues)";

```

```

SELECT fn_endereco_completo(99) AS "Endereco Cliente 99 (Inexistente)";

-- Relatorio: todos os clientes com seus enderecos
SELECT
    c.id_cliente,
    c.nome,
    fn_endereco_completo(c.id_cliente) AS endereco
FROM cliente c
ORDER BY c.id_cliente;

-- =====
-- SITUACAO 3
-- FUNCAO: fn_produto_mais_vendido()
-- Sem parametros. Identifica qual produto teve maior
-- quantidade total vendida somando todos os itens de
-- carrinho_venda. Faz JOIN com produto para retornar o nome.
-- =====

CREATE OR REPLACE FUNCTION fn_produto_mais_vendido()
RETURNS VARCHAR(60)
LANGUAGE plpgsql
AS $$
DECLARE
    v_nome VARCHAR(60);
BEGIN
    SELECT    p.nome
    INTO      v_nome
    FROM      carrinho_venda cv
    JOIN      produto      p  ON p.id_produto = cv.fk_id_produto
    GROUP BY  p.id_produto, p.nome
    ORDER BY  SUM(cv.quantidade) DESC
    LIMIT     1;

    RETURN COALESCE(v_nome, 'Nenhum produto vendido');
END;
$$;

-- TESTE - Situacao 3
SELECT fn_produto_mais_vendido() AS "Produto Mais Vendido";

-- Verificacao: ranking completo de produtos por quantidade
SELECT

```

```

        p.nome                                AS produto,
        SUM(cv.quantidade)                    AS total_vendido,
        COUNT(DISTINCT cv.fk_id_venda)        AS num_vendas
FROM carrinho_venda cv
JOIN produto p ON p.id_produto = cv.fk_id_produto
GROUP BY p.id_produto, p.nome
ORDER BY total_vendido DESC;

-- =====
-- SITUACAO 4
-- FUNCAO: fn_desconto_progressivo(p_valor, p_quantidade)
-- Calcula o total com desconto aplicado por faixa:
--   1-2 unidades: 0% de desconto
--   3-5 unidades: 5% de desconto
--   6-10 unidades: 10% de desconto
--   11+ unidades: 15% de desconto
-- Funcao pura - nao acessa tabelas.
-- =====

CREATE OR REPLACE FUNCTION fn_desconto_progressivo(
    p_valor          DECIMAL(15,2),
    p_quantidade     INT
)
RETURNS DECIMAL(15,2)
LANGUAGE plpgsql
AS $$
DECLARE
    v_percentual DECIMAL(5,2);
    v_total      DECIMAL(15,2);
BEGIN
    v_percentual := CASE
        WHEN p_quantidade <= 2 THEN 0.00
        WHEN p_quantidade <= 5 THEN 0.05
        WHEN p_quantidade <= 10 THEN 0.10
        ELSE 0.15
    END;

    v_total := p_valor * p_quantidade * (1 - v_percentual);

    RETURN ROUND(v_total, 2);
END;
$$;

```

```

-- TESTES - Situacao 4
SELECT fn_desconto_progressivo(1000.00, 2) AS "2 un - sem desconto
(R$ 2000.00)";
SELECT fn_desconto_progressivo(1000.00, 4) AS "4 un - 5% desconto
(R$ 3800.00)";
SELECT fn_desconto_progressivo(1000.00, 8) AS "8 un - 10% desconto
(R$ 7200.00)";
SELECT fn_desconto_progressivo(1000.00, 15) AS "15 un - 15% desconto
(R$ 12750.00)";

-- Aplicando a funcao nos itens reais do carrinho
SELECT
    p.nome                                AS produto,
    cv.quantidade::INT                    AS qtd,
    cv.valor_unitario                      AS preco_unit,
    cv.quantidade * cv.valor_unitario      AS sem_desconto,
    fn_desconto_progressivo(cv.valor_unitario,
                            cv.quantidade::INT) AS com_desconto,
    (cv.quantidade * cv.valor_unitario) - fn_desconto_progressivo(cv.valor_unitario,
                            cv.quantidade::INT) AS economia
FROM carrinho_venda cv
JOIN produto p ON p.id_produto = cv.fk_id_produto
ORDER BY cv.quantidade DESC
LIMIT 10;

-- =====
-- SITUACAO 5
-- FUNCAO: fn_total_gasto_cliente(p_id_cliente INT)
-- Retorna o valor total gasto por um cliente somando
-- todos os itens de todas as suas vendas.
-- Faz JOIN duplo: venda -> carrinho_venda.
-- Retorna 0.00 para cliente sem compras.
-- =====

CREATE OR REPLACE FUNCTION fn_total_gasto_cliente(p_id_cliente INT)
RETURNS DECIMAL(15,2)
LANGUAGE plpgsql
AS $$
DECLARE
    v_total DECIMAL(15,2);

```

```

BEGIN
    SELECT COALESCE(SUM(cv.quantidade * cv.valor_unitario), 0.00)
    INTO    v_total
    FROM    venda          v
    JOIN    carrinho_venda cv ON cv.fk_id_venda = v.id_venda
    WHERE   v.fk_id_cliente = p_id_cliente;

    RETURN v_total;
END;
$$;

-- TESTES - Situacao 5
SELECT fn_total_gasto_cliente(1)  AS "Total Gasto - Carlos Silva";
SELECT fn_total_gasto_cliente(4)  AS "Total Gasto - Julia Ferreira";
SELECT fn_total_gasto_cliente(99) AS "Total Gasto - Inexistente";

-- Ranking: todos os clientes com compras
SELECT
    c.nome,
    COUNT(DISTINCT v.id_venda)          AS qtd_pedidos,
    fn_total_gasto_cliente(c.id_cliente) AS total_gasto
FROM cliente c
LEFT JOIN venda v ON v.fk_id_cliente = c.id_cliente
GROUP BY c.id_cliente, c.nome
HAVING fn_total_gasto_cliente(c.id_cliente) > 0
ORDER BY total_gasto DESC;

-- =====
-- SITUACAO 6
-- FUNCAO: fn_status_estoque(p_id_produto INT)
-- Verifica o nivel de estoque de um produto comparando
-- quantidade_atual com quantidade_minima:
--     quantidade_atual = 0          -> 'SEM ESTOQUE'
--     quantidade_atual <= minima    -> 'ESTOQUE BAIXO'
--     quantidade_atual > minima     -> 'ESTOQUE OK'
--     produto inexistente          -> 'Produto nao encontrado'
-- REQUER o UPDATE de pre-requisito acima.
-- =====

CREATE OR REPLACE FUNCTION fn_status_estoque(p_id_produto INT)
RETURNS VARCHAR(40)
LANGUAGE plpgsql

```

```

AS $$
DECLARE
    v_atual    DECIMAL(15,2);
    v_minima   DECIMAL(15,2);
BEGIN
    SELECT quantidade_atual, quantidade_minima
    INTO    v_atual, v_minima
    FROM    produto
    WHERE   id_produto = p_id_produto;

    IF NOT FOUND THEN
        RETURN 'Produto nao encontrado';
    END IF;

    RETURN CASE
        WHEN v_atual = 0           THEN 'SEM ESTOQUE'
        WHEN v_atual <= v_minima THEN 'ESTOQUE BAIXO'
        ELSE                       'ESTOQUE OK'
    END;
END;
$$;

-- TESTES - Situacao 6
SELECT fn_status_estoque(1) AS "iPhone 14"      [15/5]";
SELECT fn_status_estoque(2) AS "Galaxy S23"     [3/5]";
SELECT fn_status_estoque(3) AS "PlayStation 5"  [0/2]";
SELECT fn_status_estoque(7) AS "Asus ROG Phone" [1/3]";
SELECT fn_status_estoque(99) AS "Produto 99"    [Inexistente]";

-- Relatorio geral de estoque
SELECT
    p.id_produto,
    p.nome,
    p.quantidade_atual AS atual,
    p.quantidade_minima AS minima,
    fn_status_estoque(p.id_produto) AS status
FROM produto p
ORDER BY
    CASE fn_status_estoque(p.id_produto)
        WHEN 'SEM ESTOQUE' THEN 1
        WHEN 'ESTOQUE BAIXO' THEN 2
        ELSE 3
    END,

```



```
p.nome;
```

Consultas:

Consulta 1:

Total Venda 1 (Carlos Silva)	Total Venda 99 (Inexistente)
abc Filter...	abc Filter...
37999.28	0.00

Consulta 2:

Endereco Cliente 1 (Carlos Silva)
abc Filter...
Rua Paulista, 100 - Apto 12 - Bela Vista - São Paulo - CEP: 01311000

Endereco Cliente 99 (Inexistente)
abc Filter...
Endereco nao cadastrado

Consulta 3:

Produto Mais Vendido
abc Filter...
Headset HyperX

Consulta 4:

2 un - sem desconto (R\$ 2000.00) ↑	4 un - 5% desconto (R\$ 3800.00)
abc Filter...	abc Filter...
2000.00	3800.00

8 un - 10% desconto (R\$ 7200.00)
abc Filter...
7200.00

Consulta 5:

Total Gasto - Carlos Silva ↑
abc Filter...
97201.15

Total Gasto - Inexistente
abc Filter...
0.00

Consulta 6:

iPhone 14 [15/5]
abc Filter...
ESTOQUE OK

Galaxy S23 [3/5]
abc Filter...
ESTOQUE BAIXO

PlayStation 5 [0/2] ↑
abc Filter...
SEM ESTOQUE

Produto 99 [Inexistente] ↑
abc Filter...
Produto nao encontrado